**Performance Benefit Maximization with Power Constraints.** A third formulation for the application allocation problem is to maximize the net performance benefit given a fixed power budget for each server, where the net benefit is computed as the difference between the performance benefit and migration cost.

$$maximize \sum_{i=1}^{N} \sum_{j=1}^{M} B(x_{i,j}) - Mig(A_o, A_I) \quad (3)$$

We next present the various model assumptions that $pMapper$ needs to make to solve the application placement problem.

## 3 Model Assumptions and Experimental Reality

In this section, we study the various underlying model assumptions and the feasibility of constructing estimation models required by the three formulations of the application placement problem.

Our testbed to experimentally validate the model assumptions consists of two different experimental setups. The first setup is an IBM HS-21 Bladecenter with mutiple blades. Each blade has 2 Xeon5148 dual-core Processors and runs two compute-intensive applications from an HPC suite, namely *daxpy* and $fma$ and a Linpack benchmark $HPL$ [16] on VMWare ESX Hypervisor . Each blade server has an L2 cache of 4MB, FSB of 1.33 GHz, with each processor running at 2.33 GHz. The second setup consists of 9 IBM x3650 rack servers running VMWare ESX with L2 cache of 4MB. Each server has a quad-core Xeon5160 processor running at 2.99 GHz. This setup runs the Trade6 application as well as the two HPC applications *daxpy* and *fma*. The overall system power is measured through IBM Active Energy Manager APIs [13]. We now list the key model assumptions and our experimental findings on the veracity (or the lack of it) of the assumptions. We use these findings to investigate the practicality of the optimization formulations discussed earlier.

### 3.1 Performance Isolation in Virtualized Systems

Virtualization allows applications to share the same physical server by creating multiple virtual machines in a manner such that each application can assume ownership of the virtual machine. However, such sharing is possible only if one virtual machine is isolated from other virtual machines hosted on the same physical server. Hence, we first studied this fundamental underlying assumption by running a background load using $fma$ and a foreground load using *daxpy*. The applications run on two different VMs with fixed reservations. We varied the intensity of the background load and measured the performance (throughput) of the foreground daxpy application (Fig. 2(a)).

One can observe that the *daxpy* application is isolated from the load variation in the background application. However, we conjectured that applications may only be isolated in terms of CPU and memory, while still competing for the shared cache. To validate this conjecture, we increased the memory footprint of